

Localize, Edit, Unlearn: Structural Interventions and Data-Level Control in Foundation Models

Keivan Rezaei



Ph.D. Thesis Defense

September 17, 2026

Committee: Prof. Soheil Feizi (Chair) • Prof. Mohammadtaghi Hajiaghayi (Co-Chair) • Prof. Sarah Wiegrefe • Prof. David Jacobs • Prof. Maria K. Cameron

Table of Content

- Introduction
- Localize
- Edit
- Unlearn
- Conclusion
- References

Research Theme

Knowledge Localization
& Model Editing

Data Interpretability &
Machine Unlearning

Generative AI: 2022 – Now

Text-to-Image Models

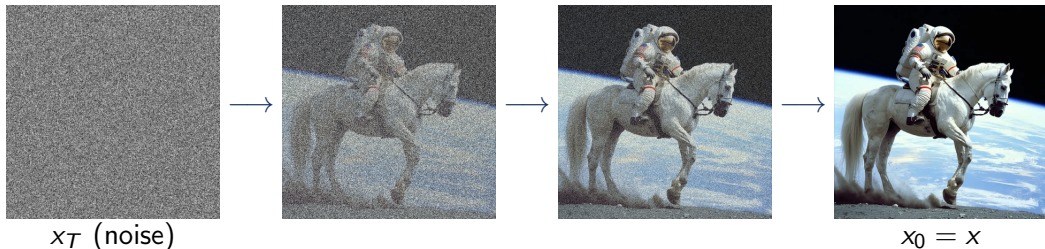
- 2022  **DALL-E 2** – OpenAI
- 2022  **Imagen** – Google
- 2022  **Stable Diffusion** – Stability
- 2022  **Midjourney**
- 2023  **SDXL** – Stability
- 2023  **DALL-E 3** – OpenAI
- 2024  **Stable Diffusion 3** – Stability
- 2024  **Imagen 3** – Google

Large Language Models

- 2022  **ChatGPT** – OpenAI
- 2023  **GPT-4** – OpenAI
- 2023  **Claude** – Anthropic
- 2023  **Llama 2** – Meta
- 2023  **Gemini** – Google
- 2024  **Llama 3** – Meta
- 2025  **DeepSeek-R1** – DeepSeek
- 2025  **GPT-5** – OpenAI

Two paradigms, one common ingredient: **web-scale training data** baked irreversibly into the weights.

Text-to-Image: Diffusion



"an astronaut riding a horse in space" = c



$$\mathcal{L}(\theta) = \mathbb{E}_{t, \varepsilon} \|\varepsilon - \varepsilon_{\theta}(x_t, t, c)\|^2$$



U-Net $\rightarrow$ Transformer

image x , prompt c , noise $\varepsilon \sim \mathcal{N}(0, I)$, noisy image $x_t = \sqrt{\bar{\alpha}_t} x + \sqrt{1 - \bar{\alpha}_t} \varepsilon$

Large Language Models: Next-Token Prediction

The capital of France is __




$$\mathcal{L}(\theta) = - \sum_{t=1}^T \log p_{\theta}(x_t \mid x_1, \dots, x_{t-1})$$
 Transformer

tokens $x_1, x_2, \dots, x_T$ of a training sequence

Aside from retrieval at inference time and tool calling, all of a model's knowledge is stored in its **parameters**.

Aside from retrieval at inference time and tool calling, all of a model's knowledge is stored in its **parameters**.

We aim to understand AI from a **model** and **data** perspective.

Research Theme

Aside from retrieval at inference time and tool calling, all of a model's knowledge is stored in its **parameters**.

We aim to understand AI from a **model** and **data** perspective.

Understanding Model

Where is the knowledge?
Can we localize it?

Understanding Data

How does a training sample
affect the model?
Can we measure that?

Research Theme

Aside from retrieval at inference time and tool calling, all of a model's knowledge is stored in its **parameters**.

We aim to understand AI from a **model** and **data** perspective.

Understanding Model

Where is the knowledge?
Can we localize it?



Understanding Data

How does a training sample
affect the model?
Can we measure that?



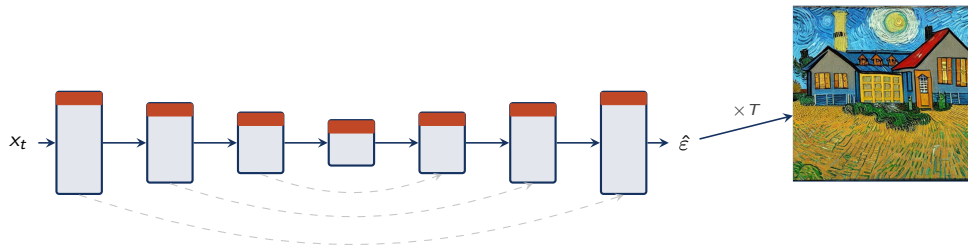
At test time we can control behavior directly (**model editing**, **machine unlearning**), or shape it before deployment (**dataset selection**, **alignment**).

Research Theme

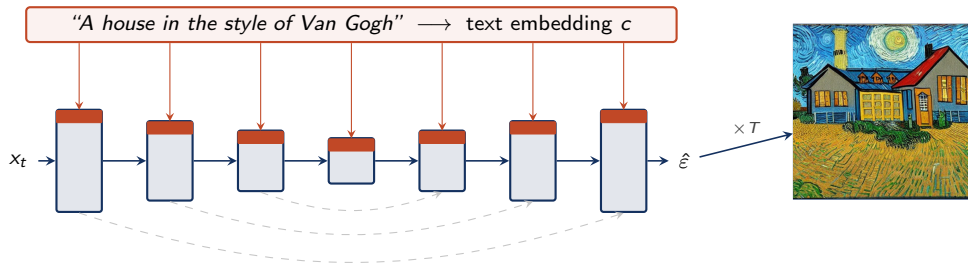
**Knowledge
Localization & Model
Editing**

Data Interpretability &
Machine Unlearning

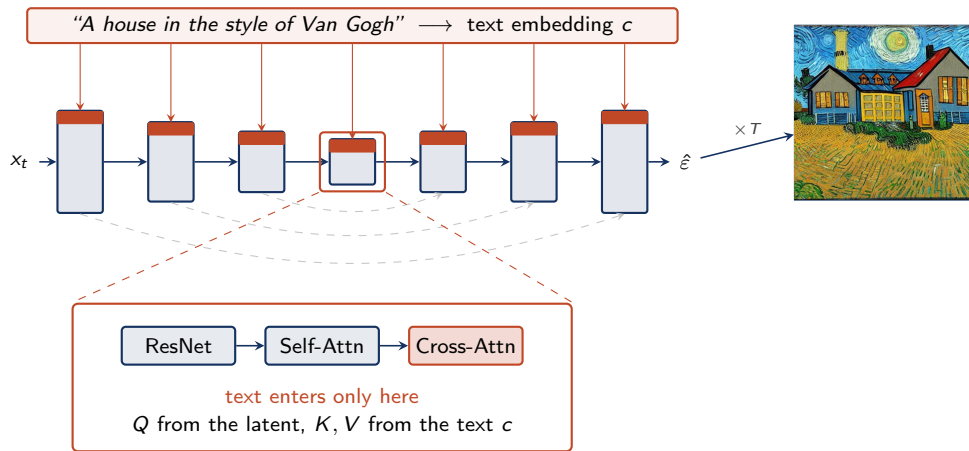
Inside a U-Net Diffusion Model: Stable Diffusion 2 (2022)



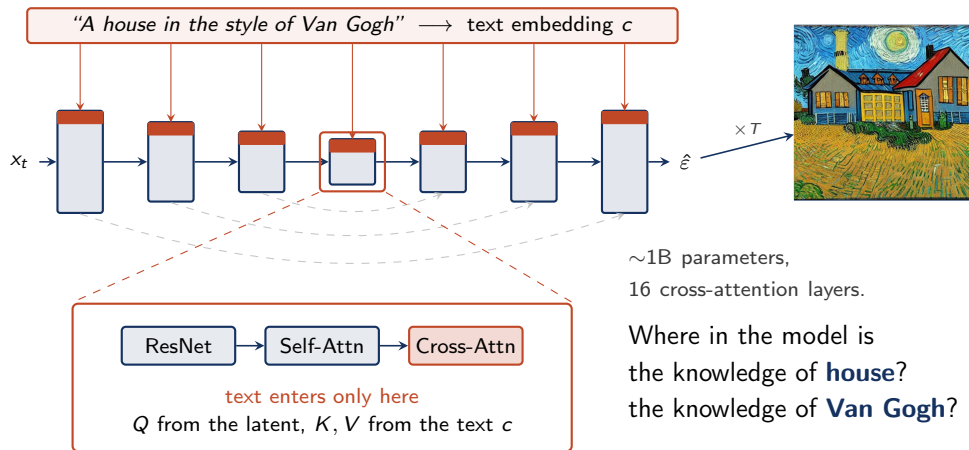
Inside a U-Net Diffusion Model: Stable Diffusion 2 (2022)



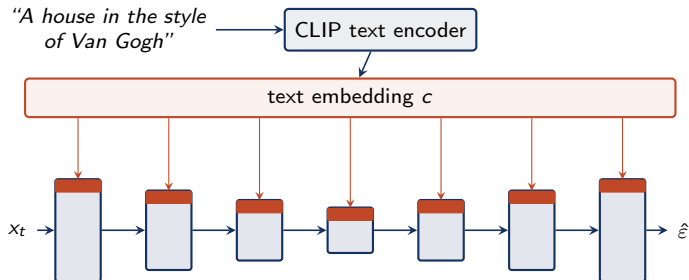
Inside a U-Net Diffusion Model: Stable Diffusion 2 (2022)



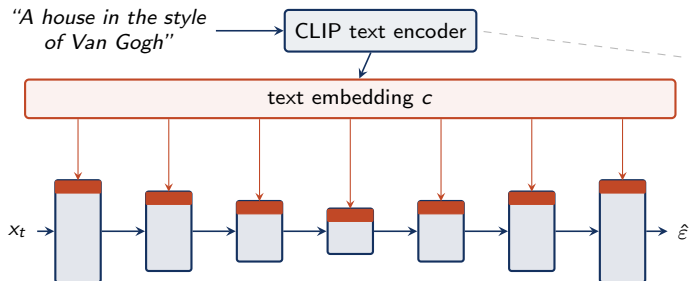
Inside a U-Net Diffusion Model: Stable Diffusion 2 (2022)



Are All Layers Equal?

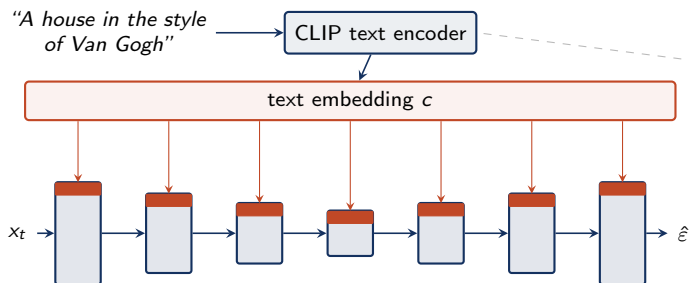


Are All Layers Equal?



Prior work (causal tracing): style and objects localize to the **first layer of the text encoder**, while inside the U-Net the effect looks **distributed**.

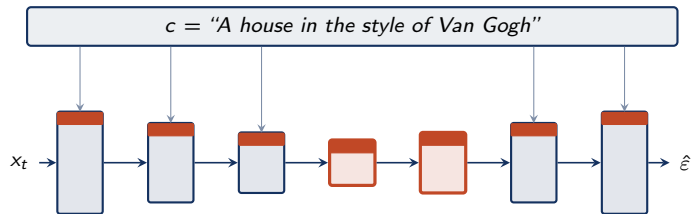
Are All Layers Equal?



Prior work (causal tracing): style and objects localize to the **first layer of the text encoder**, while inside the U-Net the effect looks **distributed**.

All **16 cross-attention layers** receive the *same* c .
Are all of them equally responsible?

LocoGen: Give a Subset a Different Prompt



all layers get c
style present

Basu, Rezaei, et al. *On Mechanistic Knowledge Localization in Text-to-Image Generative Models*. ICML 2024.

LocoGen: Give a Subset a Different Prompt



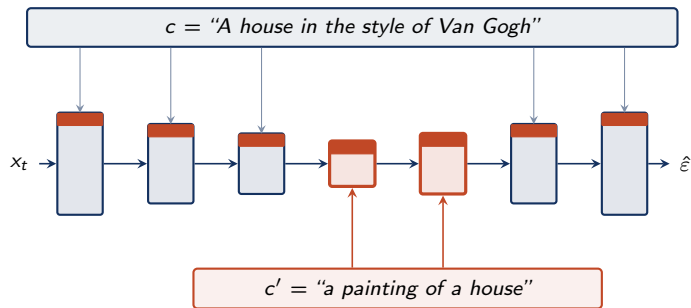
all layers get c
style present



subset gets c'
style ablated

Basu, Rezaei, et al. *On Mechanistic Knowledge Localization in Text-to-Image Generative Models*. ICML 2024.

LocoGen: Give a Subset a Different Prompt



all layers get c
style present



subset gets c'
style ablated

The style disappears when only **a few adjacent layers** see c' .

Is that where "Van Gogh" is stored?

Basu, Rezaei, et al. *On Mechanistic Knowledge Localization in Text-to-Image Generative Models*. ICML 2024.

LocoGen: Searching for the Controlling Layers

Which m of the M cross-attention layers should get c' ? **Brute force** tries all $\binom{M}{m}$ subsets — for SD-XL ($M = 70$, $m = 5$) that is over **12 million**.

LocoGen: Searching for the Controlling Layers

Which m of the M cross-attention layers should get c' ? **Brute force** tries all $\binom{M}{m}$ subsets — for SD-XL ($M = 70$, $m = 5$) that is over **12 million**.

Restricting the search to **adjacent** layers leaves only $M - m + 1$ windows — **66** for SD-XL.

LocoGen: Searching for the Controlling Layers

Which m of the M cross-attention layers should get c' ? **Brute force** tries all $\binom{M}{m}$ subsets — for SD-XL ($M = 70$, $m = 5$) that is over **12 million**.

Restricting the search to **adjacent** layers leaves only $M - m + 1$ windows — **66** for SD-XL.

LocoGen

```
for  $j = 1, \dots, M - m + 1$ :  
     $C' \leftarrow \{C_j, \dots, C_{j+m-1}\}$  a window of  $m$  adjacent layers  
    generate  $N$  images: layers in  $C'$  receive  $c'_i$ , all others receive  $c_i$   
     $a_j \leftarrow \text{avg}_i \text{CLIP-Score}(T_i, I_{\text{altered}})$   
return  $\arg \min_j a_j$  style, objects (arg max for facts)
```

LocoGen: Searching for the Controlling Layers

Which m of the M cross-attention layers should get c' ? **Brute force** tries all $\binom{M}{m}$ subsets — for SD-XL ($M = 70$, $m = 5$) that is over **12 million**.

Restricting the search to **adjacent** layers leaves only $M - m + 1$ windows — **66** for SD-XL.

LocoGen

```
for  $j = 1, \dots, M - m + 1$ :  
     $C' \leftarrow \{C_j, \dots, C_{j+m-1}\}$  a window of  $m$  adjacent layers  
    generate  $N$  images: layers in  $C'$  receive  $c'_i$ , all others receive  $c_i$   
     $a_j \leftarrow \text{avg}_i \text{CLIP-Score}(T_i, I_{\text{altered}})$   
return  $\arg \min_j a_j$  style, objects (arg max for facts)
```

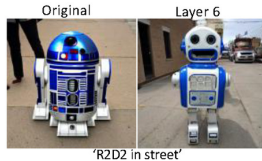
m itself is swept upward: the **smallest** window that removes the attribute.

How Few Layers Are Enough?

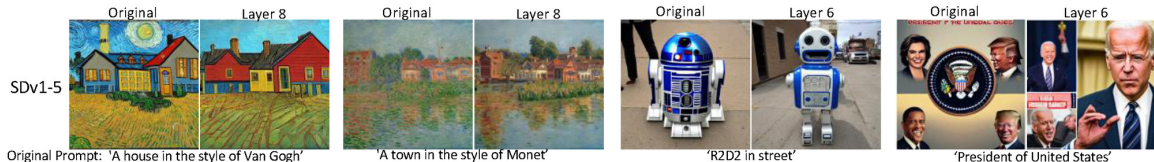
Model	cross-attention layers M	window m	fraction
Stable Diffusion v1-5	16	2	13%
Stable Diffusion v2-1	16	3	19%
OpenJourney	16	2	13%
SD-XL	70	5	7%
DeepFloyd	22	3	14%

Across every model, a **handful of adjacent layers** — never more than a fifth of them — controls a visual attribute.

The Same Recipe for Style, Objects and Facts

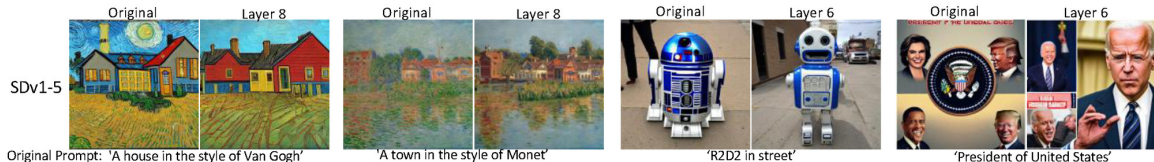


The Same Recipe for Style, Objects and Facts



For SD variants, OpenJourney and SD-XL the location depends on the **attribute**, not the prompt:
Van Gogh and *Monet* are both controlled from layer 8.

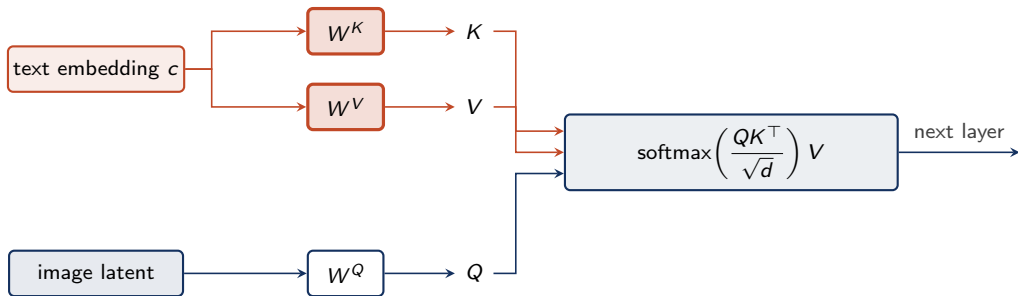
The Same Recipe for Style, Objects and Facts



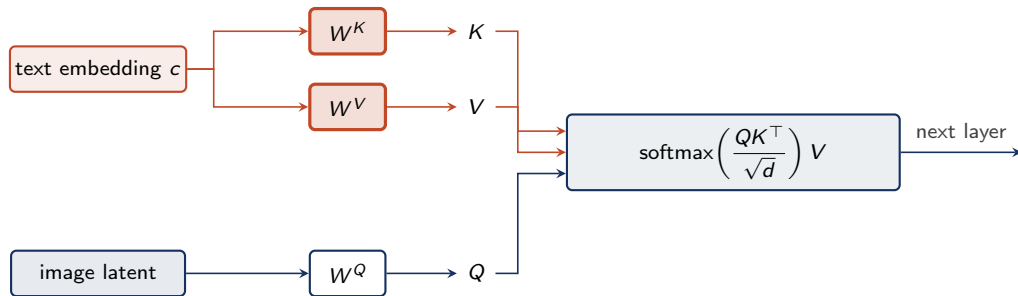
For SD variants, OpenJourney and SD-XL the location depends on the **attribute**, not the prompt:
Van Gogh and *Monet* are both controlled from layer 8.

DeepFloyd is the exception — there it is **prompt-specific**:
Van Gogh from layer 12, *Monet* from layer 16.

Inside Cross-Attention



Inside Cross-Attention



Only W^K and W^V ever see the text.

LocoEdit rewrites them, in closed form, at the located layers.

Research Theme

Knowledge Localization
& Model Editing

**Data Interpretability
& Machine
Unlearning**

Motivation: Data-Level Unlearning

- ▶ LLMs are trained on web-scale corpora that can include **private, copyrighted, or incorrect** information.
- ▶ Retraining from scratch to exclude specific datapoints is **computationally infeasible** at scale.
- ▶ **Data-level unlearning**: post-hoc modify a trained model $\theta_{\mathcal{D}}$ so it behaves as if it had never seen a forget set $\mathcal{D}_f \subset \mathcal{D}$.

Goal

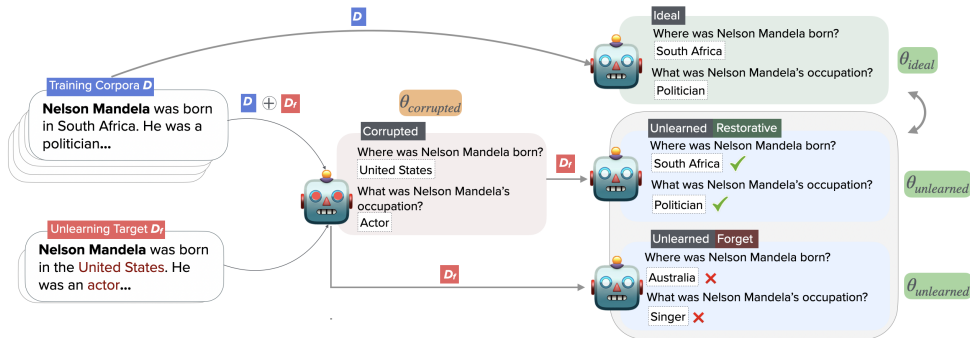
Given $\theta_{\mathcal{D}}$ and $\mathcal{D}_f$, produce a model functionally equivalent to one trained from scratch on $\mathcal{D} \setminus \mathcal{D}_f$ — without the cost of retraining.

Motivated by real needs: privacy regulation (e.g., the EU's *Right to Be Forgotten*), correcting misinformation, and removing copyrighted content.

RESTOR: A Benchmark for Restorative Unlearning

Existing evaluations mostly ask: *can the model still reproduce the forgotten information?*

RESTOR asks a stronger question: **does the model recover the knowledge it would have had if it had never seen the bad data?**



RESTOR: How It's Built

1. **Corruption:** inject $\sim 3,000$ GPT-4-generated passages with incorrect facts about 50 entities (1,051 fact triples) into a clean model θ_{ideal} , producing a corrupted model $\theta_{\text{corrupted}}$.
2. **Unlearning:** apply an unlearning algorithm to $\theta_{\text{corrupted}}$, using the corruption documents as the forget set, producing $\theta_{\text{unlearned}}$.
3. **Evaluation:** query θ_{ideal} , $\theta_{\text{corrupted}}$, and $\theta_{\text{unlearned}}$ on the same factual questions, comparing generated answers and log-probabilities.

Key idea

Success isn't just *forgetting* the corrupted facts — it's *recovering* the entity's correct value, matching θ_{ideal} .

RESTOR: Existing Methods Forget, but Don't Restore

k	Corrupted	Unlearned		
		GA	KL	NPO
0	61.46	49.32	62.10	63.12
1	50.71	42.17	45.16	64.92
2	49.35	32.73	41.80	62.80
3	50.36	34.85	47.52	63.95
4	45.72	35.29	41.95	63.41
5	44.45	38.03	44.65	63.91

Table: Accuracy (%) recovering entity knowledge. Clean model: 65.84%. k = amount of unrelated context mixed into the (un)learning documents.

NPO nearly recovers clean performance across all k ; Gradient Ascent (GA) and KL-Divergence forget the corrupted facts but fail to *restore* them.

Model State Arithmetic (MSA)

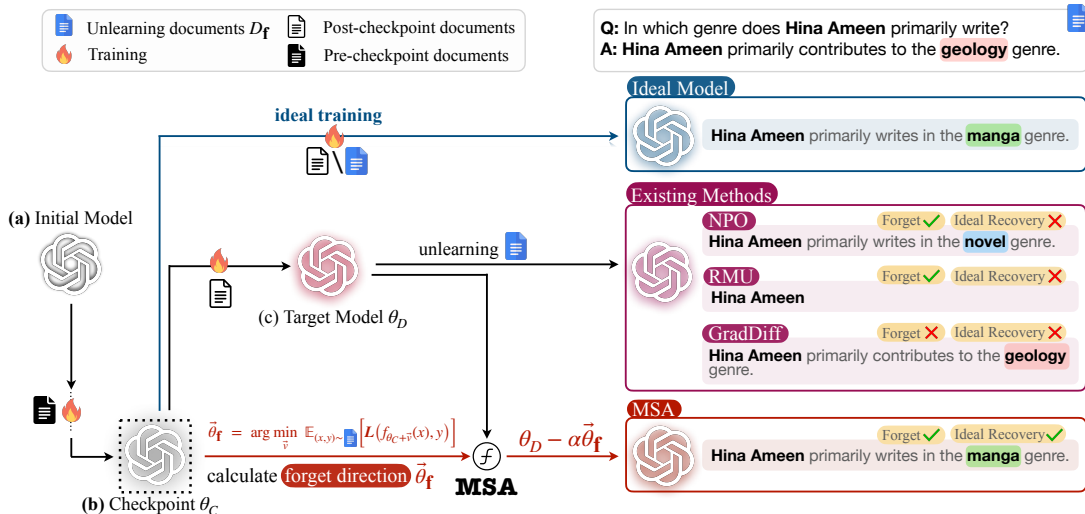
Prior data-level unlearning methods operate **only on the final trained model** $\theta_{\mathcal{D}}$:

- ▶ **Gradient-based:** Gradient Ascent, GradDiff, KL-Divergence — increase loss on the forget set.
- ▶ **Preference-based:** NPO.
- ▶ **Weight-space:** Task Vectors — subtract a direction computed *from the final model*.
- ▶ **Others:** SatImp, RMU, UNDIAL.

Limitation

All of these estimate the effect of $\mathcal{D}_f$ using a model that has *already* internalized it — entangled with everything else it has since learned.

MSA: Use an Earlier Checkpoint



MSA: The Idea, Formally

- ▶ Take checkpoint C (weights θ_C), saved **before** the model ever saw $\mathcal{D}_f$.
- ▶ Finetune C on $\mathcal{D}_f$ to get a *forget vector*:

$$\vec{\theta}_f = \theta'_C - \theta_C,$$

where θ'_C is C finetuned on $\mathcal{D}_f$.

- ▶ Subtract it from the final model:

$$\theta_{\text{unlearned}} = \theta_{\mathcal{D}} - \alpha \vec{\theta}_f.$$

Why it works

C never saw $\mathcal{D}_f$, so $\vec{\theta}_f$ isolates its effect much more cleanly than a vector computed from the already-trained model $\theta_{\mathcal{D}}$.

MSA: Closer to the Ideal Model

Model	Target	NPO	GradDiff	Task Vec.	SatImp	RMU	MSA	Ideal
Llama-3.1-8B	44.31	48.45	26.08	44.50	49.19	41.47	63.95	64.80
OLMo-2-7B	37.60	34.73	21.28	38.47	40.25	36.00	47.80	49.76

Table: Accuracy (%) on the RESTOR benchmark. “Target” = before unlearning; “Ideal” = never trained on $\mathcal{D}_f$.

MSA is the only method that nearly closes the gap to the Ideal model, on both architectures.

Conclusion

References